

A First Look at the CPU Parallel Programming Framework

PM - C++ Concurrency in Action

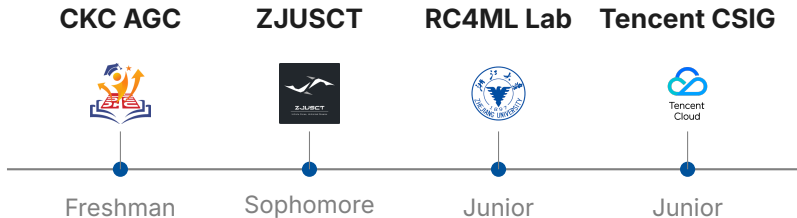
Baolin Zhu (@bowling233)

July 2, 2025

Zhejiang University Supercomputing Team

Introduction

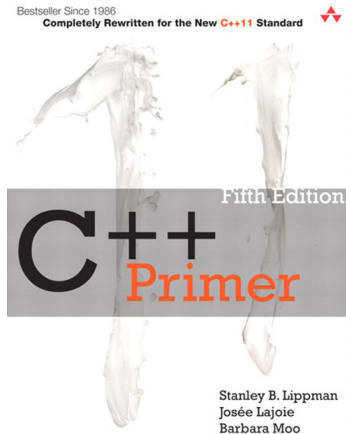
- **Baolin Zhu** (🎳 @bowling233)
- (with Eric) Leader of ZJUSCT
- **Research Interests:** Arch/OS, Three Pillars (Compute, Network and Storage)



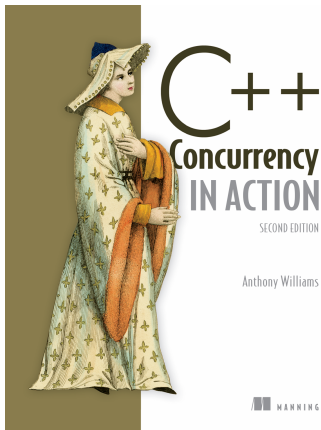
? Why C++ Concurrency?

Reducing cognitive overhead for **application-level** (rather than systems-level) programming.

- Object-Oriented Programming (OOP) & Encapsulation
- Resource Management & RAII: Smart pointers
- Templates & Generic Programming
- Standard Template Library (STL) & Abstractions
- Safer Concurrency Models
- Enhanced Type Safety & Expressiveness: References, Function Overloading, and Lambdas



- **Title:** C++ Primer (Fifth Edition)
- **Authors:** Stanley B. Lippman, Josée Lajoie, and Barbara E. Moo
- **Publisher:** Addison-Wesley Professional



- **Title:** C++ Concurrency in Action (Second Edition)
- **Authors:** Anthony Williams
- **Publisher:** Manning Publications

- **C/C++ Reference:** Use [cppreference](#) for syntax and library functions
- **Compiler Documentation:** Check your compiler's documentation for specific features and extensions (Example: [GCC Attributes](#))
- **Online Communities:** Engage with communities like Stack Overflow for troubleshooting and best practices
- **Source Codes:** You can copy from `\river\hpc101\2025\day8\_pm` to your home directory for practice

```
cp -r /river/hpc101/2025/day8_pm ~/day8_pm  
make
```

💡 Systematic vs. Problem - Driven Learning

Systematic Learning

- Builds a structured knowledge network
- Driven by internal logic of disciplines
- Strong foundation, long-term benefits
- Limitation: Time-consuming, less adaptive

Problem - Driven Learning

- Focuses on solving specific problems
- Knowledge acquisition is modular and need-based
- Quick results, highly focused
- Limitation: Fragmented knowledge, less depth

Key Insight: Effective learners adaptively combine both approaches, leveraging systematic learning for foundational understanding and problem-driven learning for practical application.

Knowledge is like a web or a tree, you should learn to start from any point on it.

1. C++ Quick Start

- Object-oriented programming
- Key differences between C++ and C
- Containers and Algorithms
- Lambda and Callable objects
- Templates and Generic Programming

2. C++ Concurrency Overview: `<thread>`, `<mutex>`, `<condition_variable>`, `<execution>`

3. Summary and Framework Selection

Prerequisites

- **C Language Proficiency:** Basic understanding of C syntax and concepts
- **Familiarity with Pointers:** Understanding of pointers, memory management, and basic data structures
- **Basic Programming Constructs:** Loops, conditionals, functions, and arrays

C++ Quick Start

C++ Quick Start

Object - Oriented Programming

Procedural vs Object - Oriented Programming

C: Procedural Programming

- Focus on functions and data
- Functions operate on data structures
- No built-in support for encapsulation or inheritance

A class defines a type **along with a collection of operations that are related to that type.**

C++: Object - Oriented Programming

- Combines data and functions into classes
- Supports encapsulation, inheritance, and polymorphism
- Provides a more modular and reusable code structure

Procedural vs Object-Oriented Programming

C Struct (Data **and** Functions)

```
struct Student {
    char name[50];
    int age;
    float gpa;
};

void print_student(struct Student s) {
    printf("Name: %s\n", s.name);
    printf("Age: %d\n", s.age);
    printf("GPA: %.2f\n", s.gpa);
}

int main() {
    struct Student alice = {"Alice", 20, 3.8};
    print_student(alice);
    return 0;
}
```

C++ Class (Data **with** Functions)

```
class Student {
private:
    string name;
    int age;
    float gpa;
public:
    Student(string n, int a, float g)
        : name(n), age(a), gpa(g) {}
    void print() {
        cout << "Name: " << name << endl;
        cout << "Age: " << age << endl;
        cout << "GPA: " << gpa << endl;
    }
};

int main() {
    Student alice("Alice", 20, 3.8);
    alice.print(); // Object calls its own method
    return 0;
}
```

Abstract Data Types

The fundamental ideas behind classes are **data abstraction** and **encapsulation**.

- **Data Abstraction:** Separation of **interface** and **implementation**
- **Encapsulation:** Enforces this separation by hiding implementation details
- Together, they define an **Abstract Data Type (ADT)**

Interface (What users see)

- Public operations users can execute
- Contract of what the class does
- Think abstractly about behavior

Implementation (Hidden details)

- Data members
- Function bodies
- Internal helper functions

Abstract Data Type: An Example

Interface (Public)

- Book(title, author, pages, count) - Constructor
- isAvailable() - Check availability
- addCopies(num) - Add copies

Users care about:

What operations are available, not how they work

```
class Book {  
private: // Implementation (Hidden)  
    string title;  
    string author;  
    int pages;  
    int count;  
public: // Interface (Visible)  
    Book(string t, string a, int p, int c);  
    bool isAvailable() const;  
    void addCopies(int num);  
};
```

Abstract Data Types: Key Benefits

For Class Designers:

- Focus on **how** the class is implemented
- Can change implementation without breaking user code
- Control access to internal data

For Class Users:

- Don't need to know **how** the type works
- Think abstractly about **what** the type does
- Use the interface without worrying about details

Class: Definition

```
class ClassName {  
public: // Public interface  
    ClassName(); // Constructor  
    ~ClassName(); // Destructor  
    void publicMethod(); // Public method  
private: // Private implementation details  
    int privateData; // Private data member  
    void privateMethod(); // Private method  
protected: // Protected members (accessible by derived classes)  
    int protectedData; // Protected data member  
};
```

Class: Instantiation

Instantiate a class to create an object

- Use the class name as a type
- Call the constructor to initialize the object
- Objects are instances of the class

```
int main() {  
    Point p(10, 20); // Create an object of Point  
    p.display();     // Call public method  
    return 0;  
}
```

Class: this pointer

A special pointer that refers to the current object

- Used inside class methods to access members of the current object
- Helps distinguish between member variables and parameters with the same name

```
class Point {  
private:  
    int x, y;  
public:  
    Point(int x, int y) {  
        this->x = x; // Use this to refer to member variable  
        this->y = y; // Use this to refer to member variable  
    }  
};
```

Rvalue Reference: A new reference type introduced in C++11

- Allows binding to temporary objects (rvalues)
- Syntax: Type&& (double ampersand)
- Enables move semantics and perfect forwarding

```
A& a_lref = a; // lvalue reference  
A&& a_rref = a; // rvalue reference
```

See: [A Brief Introduction to Rvalue References - Bjarne Stroustrup](#)

Class: Move Semantics

```
template <class T> swap(T& a, T& b)
{
    T tmp(a);    // now we have two copies of a
    a = b;       // now we have two copies of b
    b = tmp;     // now we have two copies of tmp (aka a)
}

template <class T> swap(T& a, T& b) {
    T tmp = std::move(a);
    a = std::move(b);
    b = std::move(tmp);
}
```

Class: `std::move`

`std::move`: A utility function that casts an object to an rvalue reference

- Indicates that the object can be moved
- Does not actually move anything, just changes the type

Prototype:

```
template <class T>
typename remove_reference<T>::type&&
move(T&& a)
{
    return a;
}
```

It is now up to client code to overload key functions on whether their argument is an lvalue or rvalue.

Class: Copying Objects

C++ allows copying objects using the copy constructor

- Copy constructor: when an object is passed by value or returned from a function
- Default copy constructor performs a shallow copy (member-wise copy)
- Custom copy constructor can be defined for deep copying

```
class Point {  
public:  
    // Custom copy constructor  
    Point(const Point& other) {  
        x = other.x;  
        y = other.y;  
    }  
};
```

```
Point p1(10, 20);  
Point p2 = p1; // Calls copy constructor  
p2.display(); // Displays copied point
```

Class: Moving Objects

C++11 introduced move semantics to optimize resource management

- Move constructor transfers ownership of resources from one object to another
- Avoids deep copying, improving performance for temporary objects
- Use `std::move` to indicate an object can be moved

```
class Point {  
public:  
    // Move constructor  
    Point(Point&& other) noexcept {  
        x = other.x;  
        y = other.y;  
        other.x = 0; // Leave moved-from object  
                     ↪ in a valid state  
        other.y = 0;  
    }  
};
```

```
Point p1(10, 20);  
Point p2 = std::move(p1); // Calls move  
↪ constructor  
p2.display();             // Displays moved  
↪ point
```

Class: Destructor

Destructor: A special member function called when an object goes out of scope or is deleted

- Cleans up resources (memory, file handles, etc.)
- Automatically called when the object is destroyed
- Can be defined to perform custom cleanup actions

```
class Point {  
public:  
    // Destructor  
    ~Point() {  
        cout << "Point(" << x << ", " << y << ")  
        ↪ destroyed" << endl;  
    }  
};
```

```
int main() {  
    Point p(10, 20); // Constructor called  
    p.display();     // Displays point  
    return 0;        // Destructor called  
    ↪ automatically  
}
```

Encapsulation enforces the separation of interface and implementation

- **Hide Implementation:** Users cannot access internal details
- **Control Access:** Define exactly how others can use your class
- **Maintainability:** Change implementation without breaking user code
- **Data Protection:** Prevent accidental or malicious modification

Encapsulation: Access specifiers

```
class BankAccount {  
private:    // Only this class can access  
    double balance;  
public:    // Anyone can access  
    void deposit(double amount) {  
        if (amount > 0) balance += amount; // Safe deposit  
    }  
    double getBalance() const { return balance; } // Read-only access  
protected: // This class and its children can access  
    void setBalance(double newBalance) { balance = newBalance; }  
};
```

- struct: Members are **public** by default - less encapsulation
- class: Members are **private** by default - enforces encapsulation

Inheritance: Building on Existing Code

Like a Family Tree: Children inherit traits from parents

Derived Class (Child)

Base Class (Parent)

```
class Animal {  
protected:  
    string name;  
public:  
    Animal(string n) :  
        ↪ name(n) {}  
    void eat() { cout <<  
        ↪ name << " is  
        ↪ eating\n"; }  
    virtual void makeSound()  
        ↪ { cout << name << "  
        ↪ makes a sound\n"; }  
};
```

```
class Dog : public Animal {  
public:  
    Dog(string n) : Animal(n) {} // Call parent constructor  
    void makeSound() override { // Override parent method  
        cout << name << " barks: Woof!\n";  
    }  
    void wagTail() { cout << name << " wags tail\n"; } // New method  
};  
  
int main() {  
    Dog buddy("Buddy");  
    buddy.eat(); // Inherited from Animal  
    buddy.makeSound(); // Dog's own version  
    buddy.wagTail(); // Dog-specific behavior  
    return 0;  
}
```

Polymorphism: One Interface, Many Forms

Like a Universal Remote: Same buttons, different devices

```
class Shape {
public:
    virtual double area() = 0; // Pure virtual function
    virtual ~Shape() = default;
};

class Circle : public Shape {
    double radius;
public:
    Circle(double r) : radius(r) {}
    double area() override { return 3.14159 * radius *
        ↪ radius; }
};

class Rectangle : public Shape {
    double width, height;
public:
    Rectangle(double w, double h) : width(w), height(h) {}
    double area() override { return width * height; }
};
```

```
vector<unique_ptr<Shape>> shapes;
shapes.push_back(make_unique<Circle>(5.0));
shapes.push_back(
    make_unique<Rectangle>(4.0, 6.0));
for (auto& shape : shapes) {
    cout << "Area: " << shape->area()
        << endl;
    // Calls correct method
}
```

C++ Quick Start

The Basics

I/O Streams: Compared to C Standard I/O

C Style

```
#include <stdio.h>
#include <stdlib.h>

void print_msg(void) {
    printf("Hello from C!\n");
}

int main() {
    FILE* fp = fopen("test.txt", "r");
    if (fp) {
        // Manual resource management
        char line[100];
        while (fgets(line, 100, fp) != NULL) {
            printf("%s", line);
        }
        fclose(fp);
    }
    return 0;
}
```

C++ Style

```
#include <iostream>
#include <fstream>

void print_msg() {
    std::cout << "Hello from C++!" << std::endl;
}

int main() {
    std::ifstream file("test.txt");
    if (file.is_open()) {
        // Automatic resource management
        std::string line;
        while (std::getline(file, line)) {
            std::cout << line << std::endl;
        }
    }
    return 0;
}
```

C Pointers

```
void swap_c(int* a, int* b) {  
    int temp = *a;  
    *a = *b;  
    *b = temp;  
}  
  
int main() {  
    int x = 5, y = 10;  
    swap_c(&x, &y); // Pass addresses  
    printf("x=%d, y=%d\n", x, y);  
    return 0;  
}
```

C++ References

```
void swap_cpp(int& a, int& b) {  
    int temp = a;  
    a = b;  
    b = temp;  
}  
  
int main() {  
    int x = 5, y = 10;  
    swap_cpp(x, y); // Direct pass  
    cout << "x=" << x << ", y=" << y << endl;  
    return 0;  
}
```

- References are **aliases** for existing objects: `int a; int& b = a;`
- References must be initialized and cannot be reassigned

C++ Operators and Keywords (From cplusplus.com)

Common operators						
assignment	increment decrement	arithmetic	logical	comparison	member access	other
<pre>a = b a += b a -= b a *= b a /= b a %= b a &= b a = b a ^= b a <<= b a >>= b</pre>	<pre>++a --a a++ a--</pre>	<pre>+a -a a + b a - b a * b a / b a % b ~a a & b a b a ^ b a << b a >> b</pre>	<pre>!a a && b a b</pre>	<pre>a == b a != b a < b a > b a <= b a >= b a <=> b</pre>	<pre>a[...]</pre> <pre>*a</pre> <pre>&a</pre> <pre>a->b</pre> <pre>a.b</pre> <pre>a->*b</pre> <pre>a.*b</pre>	function call <pre>a(...)</pre> comma <pre>a, b</pre> conditional <pre>a ? b : c</pre>
Special operators						
<code>static_cast</code> converts one type to another related type <code>dynamic_cast</code> converts within inheritance hierarchies <code>const_cast</code> adds or removes <code>cv</code>-qualifiers <code>reinterpret_cast</code> converts type to unrelated type C-style <code>cast</code> converts one type to another by a mix of <code>static_cast</code>, <code>const_cast</code>, and <code>reinterpret_cast</code> <code>new</code> creates objects with dynamic storage duration <code>delete</code> destructs objects previously created by the <code>new</code> expression and releases obtained memory area <code>sizeof</code> queries the size of a type <code>sizeof...</code> queries the size of a <code>pack</code> (since C++11) <code>typeid</code> queries the type information of a type <code>noexcept</code> checks if an expression can throw an exception (since C++11) <code>alignof</code> queries alignment requirements of a type (since C++11)						

A - C	D - P	R - Z
<pre>alignas (C++11) alignof (C++11) and and_eq asm atomic_cancel (TM TS) atomic_commit (TM TS) atomic_noexcept (TM TS) auto (1) (3) (4) (5) bitand bitor bool break case catch goto char8_t (C++20) char16_t (C++11) char32_t (C++11) class (1) compl concept (C++20) const constexpr (C++20) (5) constexpr (C++11) (3) constinit (C++20) const_cast continue contract_assert (C++26) co_await (C++20) co_return (C++20) co_yield (C++20)</pre>	<pre>decltype (C++11) (2) default (1) delete (1) do double dynamic_cast else enum (1) explicit export (1) (4) extern (1) false float for (1) friend goto if (3) (5) inline (1) (3) int (1) long mutable (1) namespace new noexcept (C++11) not not_eq nullptr (C++11) operator (1) or or_eq private (4) protected public</pre>	<pre>constexpr (reflection TS) register (3) reinterpret_cast requires (C++20) return short signed sizeof (1) static static_assert (C++11) static_cast struct (1) switch synchronized (TM TS) template this (5) thread_local (C++11) throw (3) (4) true try typedef typeid typename (3) (4) union unsigned using (1) (4) virtual void volatile wchar_t while xor xor_eq</pre>

We'll meet some new friends in this class:

- **I/O:** `>>`, `<<`
- **Memory:** `new`, `delete`, `new[]`, `delete[]`
- **Type System:** `auto`, `decltype`, `using`, `operator`
- **Class:** `::`, `public`, `private`, `protected`, `friend`, `virtual`, `override`, `final`

Type	Input	Output	Both
standard	cin	cout, cerr, clog	
File	ifstream	ofstream	fstream
String	istringstream	ostringstream	stringstream

Benefits of C++ Streams:

- Type safety and flexibility
- Easier to read and maintain code
- Automatic resource cleanup when objects go out of scope

String Streams: Like a string that can be read/written like a file

```
#include <sstream>
#include <iostream>

void example_string_stream() {
    std::stringstream ss;
    ss << "The answer is " << 42;
    std::string result = ss.str();
    std::cout << result << std::endl;
}
```

Benefits:

- Concise syntax for formatting strings
- No need for manual memory management
- Can be used with any type that supports stream operators

Namespaces

- C++ uses namespaces to avoid **name collisions**
- **Standard library** functions and classes are in the `std` namespace
- Use `using namespace std;` to avoid prefixing with `std::` (should be avoided in header files and large projects)

Example:

```
#include <iostream>
using namespace std;
cout << "Hello, World!" << endl;
```

Class: Operator overloading

- C++ lets us define what the operators mean **when applied to objects of class type**.
- They are **special functions** with concise syntax
- Can be **overloaded** to provide custom behavior for user-defined types
- In a user-defined operator overload, **any type** can be used as return type (including void)
- Operators for **primitive types** like `int`, `float`, etc. cannot be overloaded

Syntax:

```
<return_type> operator<operator_symbol>(<parameters>) { /* Implementation */ }  
// An illustration of overloading the + operator  
int operator+(int a, int b) { return a + b; }
```

Class: Operator overloading

Why Overload I/O Operators?

- Makes it easy to print custom objects
- Uses existing syntax (<< and >>) for a natural feel

Prototype

```
template< class Traits >
basic_ostream<char, Traits>&
    operator<<( basic_ostream<char, Traits>& os, const char* s );
basic_istream& operator>>( int& value );
```

Example Usage:

```
cout << "Hello, World!" << endl;
cin >> user_input;
```

Function Overloading: Same Name, Different Parameters

Function Overloading: Multiple functions with the same name but different parameters

- C++ allows multiple functions with the same name
- Functions must differ in number or type of parameters
- Compiler chooses the best match based on arguments
- Return type alone is **not** enough to distinguish functions

Function Overloading vs C Naming Conventions

C Style: Different Names

```
#include <stdio.h>
#include <math.h>

// Need different names for each type
int abs_int(int x) { return x < 0 ? -x : x; }
double abs_double(double x) { return x < 0.0 ? -x : x; }
float abs_float(float x) { return x < 0.0f ? -x : x; }

int main() {
    printf("%d\n", abs_int(-5));
    printf("%.2f\n", abs_double(-3.14));
    printf("%.2f\n", abs_float(-2.5f));
    return 0;
}
```

C++ Style: Overloaded Functions

```
#include <iostream>
#include <cmath>
using namespace std;
// Same name, different parameters
int abs(int x) { return x < 0 ? -x : x; }
double abs(double x) { return x < 0.0 ? -x : x; }
float abs(float x) { return x < 0.0f ? -x : x; }

int main() {
    cout << abs(-5) << endl;    // Calls abs(int)
    cout << abs(-3.14) << endl; // Calls abs(double)
    cout << abs(-2.5f) << endl; // Calls abs(float)
    return 0;
}
```

Function Overloading: Rules and Best Practices

Overloading Rules:

- Functions must differ in **number** or **type** of parameters
- **const** and non-const parameters are considered different
- Reference and non-reference parameters are different
- Return type alone cannot distinguish overloaded functions

Example of Valid Overloads:

```
void func(int x);           // Version 1
void func(double x);        // Version 2: different type
void func(int x, int y);    // Version 3: different number of para
void func(const int& x);     // Version 4: different parameter form
// void func(int x) { return 5; } // ERROR: Only return type differs
```

C++ Quick Start

Containers and Algorithms

Standard Template Library (STL):

- **Containers:** Ready-made data structures (like toolboxes)
- **Algorithms:** Common operations (like tools)
- **Iterators:** Ways to access container elements (like handles)

Why use STL instead of arrays?

- **Safety:** Automatic bounds checking and memory management
- **Convenience:** Built-in functions for common operations
- **Performance:** Optimized implementations
- **Compatibility:** Works seamlessly with C++ features

Common Containers: Your Data Storage Options

Vector: Dynamic Array

```
vector<int> numbers;
// Add elements
numbers.push_back(10);
numbers.push_back(20);
numbers.push_back(30);
// Access elements
cout << "First: " << numbers[0] << endl;
cout << "Size: " << numbers.size() << endl;
// Iterate through all
for (int num : numbers) { cout << num << " "; }
```

Map: Key - Value Storage

```
map<string, int> scores;
// Add key-value pairs
scores["Alice"] = 95;
scores["Bob"] = 87;
scores["Charlie"] = 92;
// Look up values
cout << "Alice's score: "
      << scores["Alice"] << endl;
// Check if key exists
if (scores.find("David") != scores.end()) {
    cout << "David found\n";
} else {
    cout << "David not found\n";
}
```

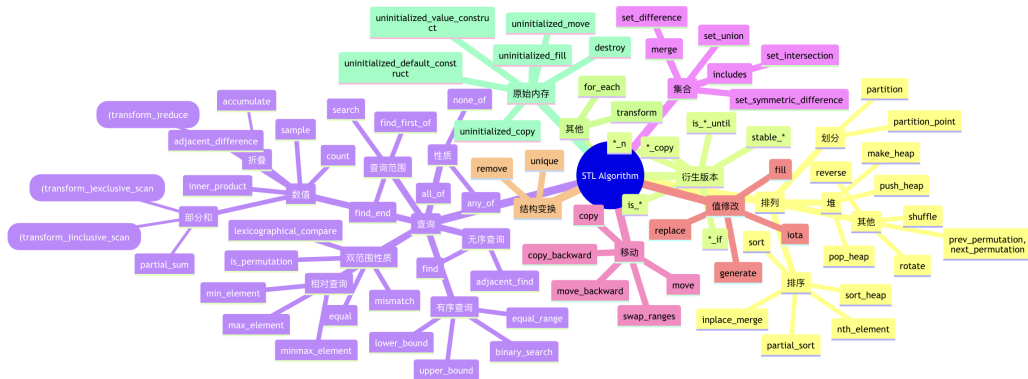
Common algorithms you'll use:

Category	Algorithms
Sorting	<code>sort</code> , <code>stable_sort</code> , <code>partial_sort</code> , <code>nth_element</code>
Searching	<code>find</code> , <code>binary_search</code> , <code>lower_bound</code> , <code>upper_bound</code>
Modifying	<code>copy</code> , <code>swap</code> , <code>replace</code> , <code>fill</code> , <code>transform</code>
Set Operations	<code>set_union</code> , <code>set_intersection</code> , <code>set_difference</code>
Numeric	<code>accumulate</code> , <code>inner_product</code> , <code>adjacent_difference</code>

Algorithms: Ready - Made Solutions

```
vector<int> numbers = {64, 34, 25, 12, 22, 11, 90};
// Sort the vector
sort(numbers.begin(), numbers.end());
cout << "Sorted: ";
for (int n : numbers) cout << n << " ";
cout << endl;
// Find an element
auto it = find(numbers.begin(), numbers.end(), 25);
if (it != numbers.end()) {
    cout << "Found 25 at position: "
         << (it - numbers.begin()) << endl;
}
// Count elements greater than 30
int count = count_if(numbers.begin(), numbers.end(),
    [](int n) { return n > 30; });
cout << "Numbers > 30: " << count << endl;
```

More on STL Algorithms



CppCon 2018: 105 STL Algorithms in Less Than an Hour

cppcon | 2018
THE C++ CONFERENCE • BELLEVUE, WASHINGTON

105 STL ALGORITHMS IN LESS THAN AN HOUR

Agenda

- Brief chat about STL algorithms
- The STL algorithms themselves

@JoBoccara

JONATHAN BOCCARA

105 STL Algorithms in
Less Than an Hour

CppCon.org

Containers vs C Arrays: The Upgrade

C Arrays (Manual Everything)

```
int arr[] = {1, 3, 2};
int size = 0;
// must write your compare function
qsort(ints, size, sizeof(int), compare_ints);
for (int i = 0; i < size; i++) {
    printf("%d ", arr[i]);
}
```

C++ Containers (Automatic)

```
// Dynamic size, automatic memory
vector<int> arr = {1, 3, 2};
sort(arr.begin(), arr.end());
for (int num : arr) {
    cout << num << " ";
}
```

Benefits: Safer, shorter, and often faster code!

C++ Quick Start

Essential Features for Concurrency

Function Call Operator

```
function (arg1, arg2, arg3,...)
```

Callable Object

- An object that can be called like a function
- Functions and pointers to functions, **lambdas**, objects created by `bind`, and **classes that overload the function-call operator**.

Lambda Expressions: The Core of Thread Functions

C Function Pointers

```
#include <pthread.h>

void* thread_func(void* arg) {
    int id = *(int*)arg;
    printf("Thread %d running\n", id);
    return NULL;
}

int main() {
    pthread_t thread;
    int id = 1;
    pthread_create(&thread, NULL,
                  thread_func, &id);
    pthread_join(thread, NULL);
    return 0;
}
```

C++ Lambda

```
#include <thread>
#include <iostream>

int main() {
    int id = 1;

    // Lambda expression
    auto thread_func = [id]() {
        cout << "Thread " << id
              << " running" << endl;
    };

    thread t(thread_func);
    t.join();

    return 0;
}
```

Lambda Expressions: Syntax

```
auto lambda_name = [capture](parameters) -> return_type {  
    // function body  
};
```

Example:

```
auto add = [](int a, int b) -> int {  
    return a + b;  
};  
cout << "Sum: " << add(3, 4) << endl; // Outputs 7
```

Lambda Expressions: Capture Clause

- `[capture]`: How to access variables from the surrounding scope
- `[=]`: Capture all by value
- `[&]`: Capture all by reference
- `[id]`: Capture specific variable by value
- `[&id]`: Capture specific variable by reference

Smart Pointers vs Manual Memory Management

C++ new/delete

C malloc/free

```
typedef struct {  
    int data;  
} Resource;  
// Manual allocation  
Resource* ptr = malloc(sizeof(Resource));  
if (!ptr) return -1;  
ptr->data = 42;  
printf("Data: %d\n", ptr->data);  
// Must remember to free  
free(ptr);  
// ptr is now dangling!
```

```
class Resource {  
    int data;  
};  
// Manual allocation  
Resource* ptr = new Resource(42);  
cout << "Data: " << ptr->getData() << endl;  
// Must remember to delete  
delete ptr;  
// ptr is now dangling!
```

C++ Smart Pointers

```
// Automatic memory management  
auto ptr = make_unique<Resource>(42);  
cout << "Data: " << ptr->getData() << endl;  
// Automatic cleanup when out of scope  
// No manual delete needed!
```

Auto Type Deduction: Let the Compiler Figure It Out

```
auto x = 42;           // int
auto y = 3.14;         // double
auto str = "Hello";    // const char*
vector<int> vec{1,2,3};
auto it = vec.begin(); // vector<int>::iterator
// Complex types made simple
map<string, int> scores;
auto map_it = scores.find("Alice"); // map<string, int>::iterator
// Range-based for loop
for(auto& pair : scores) { // std::pair<const string, int>
    cout << pair.first << ": " << pair.second << endl;
}
```

- **Less typing:** Compiler deduces the type
- **Maintainable:** Change type in one place
- **Generic:** Works with complex template types

C++ Quick Start

Templates: Generic Programming

Templates: Write Once, Use for Any Type

Problem: Writing the same function for different types

Without Templates

```
int max_int(int a, int b) {  
    return a > b ? a : b;  
}  
double max_double(double a, double b) {  
    return a > b ? a : b;  
}  
string max_string(string a, string b) {  
    return a > b ? a : b;  
}  
// Need separate function for each type!
```

With Templates

```
template<typename T>  
T max_value(T a, T b) {  
    return a > b ? a : b;  
}  
  
// Usage - compiler generates code automatically  
int result1 = max_value(10, 20);           // T = int  
double result2 = max_value(3.14, 2.71);    // T =  
↳ double  
string result3 = max_value("hello", "world"); // T  
↳ = string
```

Benefits: Code reuse, type safety, performance (no runtime overhead)

Function Templates: Generic Functions

Syntax:

```
template<typename T>
return_type function_name(parameters) {
    // function body
}
```

Class Templates: Generic Classes

Example: Generic Stack

```
template<typename T>
class Stack {
private:
    vector<T> elements;
public:
    void push(const T& item) {
        elements.push_back(item);
    }
    T pop() {
        if (elements.empty()) {
            throw runtime_error("Stack is empty");
        }
        T top = elements.back();
        elements.pop_back();
        return top;
    }
    bool empty() const {
        return elements.empty();
    }
};
```

Class Templates: Usage

```
int main() {  
    // Stack of integers  
    Stack<int> int_stack;  
    int_stack.push(10);  
    int_stack.push(20);  
    cout << "Popped: " << int_stack.pop() << endl; // Output: 20  
  
    // Stack of strings  
    Stack<string> string_stack;  
    string_stack.push("hello");  
    string_stack.push("world");  
    cout << "Popped: " << string_stack.pop() << endl; // Output: world  
  
    // Stack of custom objects  
    Stack<Student> student_stack;  
    student_stack.push(Student("Alice", 20, 3.8));  
  
    return 0;  
}
```

The Standard Template Library uses templates extensively:

```
// All STL containers are templates
vector<int> numbers;           // vector of integers
vector<string> names;         // vector of strings
vector<Student> students;     // vector of custom objects

map<string, int> scores;       // map from string to int
map<int, vector<string>> groups; // map from int to vector of strings

// STL algorithms work with any container
vector<int> nums = {3, 1, 4, 1, 5};
sort(nums.begin(), nums.end());

list<string> words = {"hello", "world", "cpp"};
auto it = find(words.begin(), words.end(), "world");
```

This is why STL is so powerful and flexible!

Templates: The Power of Generic Programming

Templates are essential for libraries like CUTLASS (CUDA Templates for Linear Algebra Subroutines):

- **Performance:** Templates allow for compile-time optimizations
- **Flexibility:** Write code that works with any type
- **Reusability:** Create generic algorithms and data structures

Example: CUTLASS uses templates to define matrix multiplication:

```
template<typename T>
void matrix_multiply(const T* A, const T* B, T* C, int M, int N, int K) {
    for (int i = 0; i < M; ++i) {
        for (int j = 0; j < N; ++j) {
            C[i * N + j] = 0;
            for (int k = 0; k < K; ++k) {
                C[i * N + j] += A[i * K + k] * B[k * N + j];
            }
        }
    }
}
```


Disadvantages of using templates

```
q++;
```

[illegible]

```

unresolved external symbol "public: __cdecl Settings::Settings(class
std::basic_string<char>, struct std::traits<char>, class std::allocator<
char> >, class std::map<class std::basic_string<char>, struct std::traits<
char>, class std::allocator<char> >, class std::basic_string<char>, struct
std::traits<char>, class std::allocator<char> >, struct std::less<class
std::basic_string<char>, struct std::traits<char>, class std::allocator<
char> >, class std::allocator<struct std::pair<class std::basic_string<char>, struct
std::traits<char>, class std::allocator<char> >>, const class
std::basic_string<char>, struct std::traits<char>, class std::allocator<
char> >, const class std::map<class std::basic_string<char>, struct std::traits<
char>, class std::allocator<char> >, class std::allocator<struct std::pair<class
std::basic_string<char>, struct std::traits<char>, class std::allocator<
char> >, const class std::map<class std::basic_string<char>, struct std::traits<
char>, class std::allocator<char> >, class std::allocator<char> >, struct
std::traits<char>, class std::allocator<char> >)<".

```

PM - C++ Concurrency in Action | C++ Quick Start • 58/127

C++ Quick Start

Concurrency Support

- **C++11** (We will learn):
 - `<thread>`: Create and manage threads
 - `<mutex>`: Mutual exclusion for shared data
 - `<atomic>`: Atomic operations for thread safety
 - `<condition_variable>`: Synchronization between threads
 - `<future>`: Asynchronous results
- **C++20**:
 - `<stop_token>`: Cooperative cancellation
 - `<semaphore>`, `<latch>`, and `<barrier>`: Advanced synchronization

<thread>

<thread>

Basic Thread Management

C++ Thread Constructor:

```
template< class F, class... Args >
    explicit thread( F&& f, Args&&... args );
thread( thread&& other ) noexcept; // Move constructor
```

- other: another thread object to construct this thread object with
- f: Callable object to execute in the new thread
- args: Arguments to pass to the new function

Launching a Thread - Basic Examples

Simple Function

```
#include <thread>
#include <iostream>

void hello() {
    cout << "Hello from thread!"
         << endl;
}

int main() {
    thread t(hello);
    t.join();
    return 0;
}
```

Function with Parameters

```
#include <thread>
#include <iostream>

void worker(int id, const string& name) {
    cout << "Worker " << id
         << " (" << name << ") running"
         << endl;
}

int main() {
    thread t(worker, 42, "Alice");
    t.join();
    return 0;
}
```

Launching a Thread - Function Objects and Lambdas

Function Object

```
#include <thread>
#include <iostream>

class background_task {
public:
    void operator()() {
        cout << "Background task running"
              << endl;
    }
};

int main() {
    background_task task;
    thread t(task);
    t.join();
    return 0;
}
```

Lambda Expression

```
#include <thread>
#include <iostream>

int main() {
    thread t([]() {
        cout << "Lambda task running"
              << endl;
    });

    t.join();
    return 0;
}
```

! Most Vexing Parse Problem

Common Pitfall: C++ interprets ambiguous syntax as function declarations

```
struct background_task {  
    void operator>()() {  
        // do work  
    }  
};  
  
// WRONG! This declares a function, not creates a thread!  
thread my_thread(background_task());  
  
// Correct ways:  
thread t1{background_task()};    // uniform initialization  
thread t2((background_task()));  // extra parentheses  
thread t3(background_task{});    // temporary object
```

Recommended: Use uniform initialization `{}` or lambdas to avoid parsing issues

Waiting for a Thread to Complete

join(): Wait for thread completion before continuing

```
void worker(int seconds) {
    this_thread::sleep_for(
        chrono::seconds(seconds));
    cout << "Work done!" << endl;
}

int main() {
    cout << "Starting thread..." << endl;
    thread t(worker, 2);
    cout << "Waiting for completion..."
        << endl;
    t.join(); // Block until thread finishes
    cout << "Thread completed!" << endl;
    return 0;
}
```

Basic join() Example

```
void worker(int id) {
    cout << "Worker " << id
        << " running" << endl;
}

int main() {
    vector<thread> threads;
    for(int i = 0; i < 4; i++) {
        threads.emplace_back(worker, i);
    }
    for(auto& t : threads) {
        t.join();
    }
    cout << "All threads completed!"
        << endl;
    return 0;
}
```

Multiple Threads

Waiting in Exceptional Circumstances

Problem: Exception-safe thread management with RAI

```
void do_work() {  
    thread t([]() {  
        // long running task  
    });  
  
    // If exception thrown here,  
    // t.join() won't be called!  
    risky_operation();  
  
    t.join(); // May not be reached  
}
```

Without RAI - Dangerous!

```
class thread_guard {  
    thread& t;  
public:  
    explicit thread_guard(thread& t_) : t(t_) {}  
    ~thread_guard() { if(t.joinable()) { t.join(); } }  
    thread_guard(const thread_guard&) = delete;  
    thread_guard& operator=(const thread_guard&) = delete;  
};  
void do_work() {  
    thread t([]() { /* long running task */ });  
    thread_guard g(t);  
    risky_operation(); // Safe!  
    // Destructor ensures join()  
}
```

With RAI - Safe!

Running Threads in the Background

detach(): Run thread independently without waiting

```
thread t(background_task);  
// Detach - thread runs independently  
t.detach();  
  
cout << "Main continues..." << endl;  
// Main might end before background finishes!  
this_thread::sleep_for(  
    chrono::seconds(3));
```

```
thread t(worker);  
// Check if thread can be joined  
if (t.joinable()) {  
    cout << "Thread is joinable" << endl;  
    t.join();  
}  
// After join/detach, not joinable  
cout << t.joinable() << endl; // false  
thread t2(worker);  
t2.detach();  
cout << t2.joinable() << endl; // false
```

Detached Thread Example

Checking Thread State

Important: Every thread must be either joined or detached before destruction!

<thread>

Passing Arguments to a Thread Function

Parameter Passing Basics

Key Principle: Thread constructor copies arguments as rvalues

String Conversion Issues

```
void print_string(const string& str) {  
    cout << str << endl;  
}  
  
// UNSAFE! Pointer passed, conversion happens in new thread  
thread unsafe_executor() {  
    char buffer[] = "Hello World";  
    return thread(print_string, buffer);  
}  
  
// SAFE! Force conversion before thread creation  
thread safe_executor() {  
    char buffer[] = "Hello World";  
    return thread(print_string, string(buffer));  
}
```

```
try {  
    thread t1 = unsafe_executor();  
    t1.join();  
} catch (const exception& e) {  
    cerr << "Error in unsafe_executor:  
    ↳ " << e.what() << endl;  
}  
thread t2 = safe_executor();  
t2.join();
```

Member Functions

```
class X {  
public:  
    void do_work(int n) {  
        cout << "Work " << n  
              << " done by object" << endl;  
    }  
};  
  
int main() {  
    X my_x;  
  
    // Pass member function and object  
    thread t(&X::do_work, &my_x, 42);  
  
    t.join();  
    return 0;  
}
```

<thread>

Transferring Ownership of a Thread

Move Semantics with Threads

thread is move-only: Cannot be copied, only moved

```
thread t1(worker);  
// Move constructor  
thread t2 = move(t1);  
// t1 is now empty  
  
// Move from function  
thread t3 = create_thread();  
  
// t1.join(); // ERROR! t1 is empty  
t2.join();  
t3.join();
```

Basic Move Operations

```
thread t1(task1);  
thread t2(task2);  
  
// Join first thread  
t1.join();  
  
// Move assign - t1 takes over t2  
t1 = move(t2);  
// t2 is now empty  
  
t1.join();
```

Move Assignment

Transferring Ownership with Move - Advanced

Move Semantics: Transfer unique resources to threads

```
void process_data(unique_ptr<int> ptr) {  
    if (ptr) {  
        cout << "Processing: " << *ptr << endl;  
        *ptr *= 2;  
        cout << "Result: " << *ptr << endl;  
    }  
}  
  
int main() {  
    auto ptr = make_unique<int>(42);  
  
    // Transfer ownership to thread using move  
    thread t(process_data, move(ptr));  
  
    // ptr is now nullptr, ownership transferred  
    assert(ptr == nullptr);  
  
    t.join();  
    return 0;  
}
```

Managing Multiple Threads with Move Semantics

```
class scoped_thread {  
    thread t;  
public:  
    explicit scoped_thread(thread t_) :  
        ↪ t(move(t_)) {  
        if (!t.joinable()) {  
            throw logic_error("No thread");  
        }  
    }  
  
    ~scoped_thread() {  
        t.join();  
    }  
  
    scoped_thread(const scoped_thread&) = delete;  
    scoped_thread& operator=(const scoped_thread&)  
        ↪ = delete;  
};
```

```
int main() {  
    vector<scoped_thread> threads;  
  
    for (int i = 0; i < 3; ++i) {  
        threads.emplace_back(thread(worker, i));  
    }  
  
    // Threads automatically joined when  
    ↪ scoped_thread destructors run  
    return 0;  
}
```

<thread>

Choosing the Number of Threads at Runtime

`thread::hardware_concurrency()`: Get optimal thread count

```
unsigned int num_threads = thread::hardware_concurrency();

if (num_threads == 0) {
    num_threads = 2; // Fallback if not available
}

cout << "Using " << num_threads << " threads" << endl;

vector<thread> threads;
for (unsigned int i = 0; i < num_threads; ++i) {
    threads.emplace_back(worker, i);
}

for (auto& t : threads) {
    t.join();
}
```

Parallel Accumulate - Helper Function

Step 1: Define the worker function for each thread

```
template<typename Iterator, typename T>
void accumulate_block(Iterator first, Iterator last, T& result) {
    result = accumulate(first, last, result);
}
```

Function Purpose:

- Each thread processes a block of data
- Computes sum of elements from `first` to `last`
- Stores result in reference parameter
- Uses standard library `accumulate` function

Next: Calculate optimal thread configuration

Parallel Accumulate - Thread Configuration

Step 2: Calculate optimal number of threads and work distribution

```
template<typename Iterator, typename T>
T parallel_accumulate(Iterator first, Iterator last, T init) {
    unsigned long const length = distance(first, last);
    if (!length) return init;

    // Configuration parameters
    unsigned long const min_per_thread = 25;
    unsigned long const max_threads =
        (length + min_per_thread - 1) / min_per_thread;

    unsigned long const hardware_threads =
        thread::hardware_concurrency();

    unsigned long const num_threads =
        min(hardware_threads != 0 ? hardware_threads : 2, max_threads);

    unsigned long const block_size = length / num_threads;
    // ...
}
```

Parallel Accumulate - Thread Creation & Execution

Step 3: Create threads and distribute work

```
// Storage for results and threads
vector<T> results(num_threads);
vector<thread> threads(num_threads - 1);
// Create worker threads (all but last)
Iterator block_start = first;
for (unsigned long i = 0; i < (num_threads - 1); ++i) {
    Iterator block_end = block_start;
    advance(block_end, block_size);
    threads[i] = thread(accumulate_block<Iterator, T>,
                       block_start, block_end, ref(results[i]));
    block_start = block_end;
}
// Main thread handles the last block
accumulate_block(block_start, last, results[num_threads - 1]);
// Wait for all threads and combine results
for (auto& entry : threads) entry.join();
return accumulate(results.begin(), results.end(), init);
```

<thread>

Identifying Threads

Thread IDs - Basic Usage

thread::id: Unique identifier for each thread

```
void worker() {  
    cout << "Thread ID: " << this_thread::get_id() << endl;  
}  
cout << "Main thread ID: " << this_thread::get_id() << endl;  
thread t(worker);  
cout << "Worker thread ID: " << t.get_id() << endl;  
t.join();  
cout << "After join: " << t.get_id() << endl; // Default ID
```

Key Functions:

- `this_thread::get_id()`: Get current thread's ID
- `thread_obj.get_id()`: Get thread object's ID
- Default-constructed ID represents "no thread"

Thread ID Characteristics:

- **Unique:** Each executing thread has a unique ID
- **Comparable:** Can be compared for equality and ordering
- **Hashable:** Can be used as keys in containers
- **Default-constructible:** Default ID represents "no thread"

```
// Thread ID comparison
thread::id main_id = this_thread::get_id();
thread::id default_id; // Default-constructed
cout << "IDs equal: " << (main_id == default_id) << endl; // false
// Use as container key
unordered_set<thread::id> thread_set;
thread_set.insert(main_id);
// Useful for debugging and thread identification
map<thread::id, string> thread_names;
thread_names[this_thread::get_id()] = "main_thread";
```

Best Practices Summary

Thread Management:

- Always `join()` or `detach()` threads before destruction
- Use `RAII` (e.g., `thread_guard`) for exception safety
- Prefer move semantics for thread ownership transfer

Parameter Passing:

- Be careful with string literals - convert to `std::string` explicitly
- Use `std::ref()` for reference parameters
- Move unique resources (e.g., `unique_ptr`) when transferring ownership

Performance:

- Use `hardware_concurrency()` to determine optimal thread count
- Balance thread overhead vs. parallelism (minimum work per thread)
- Consider thread pool patterns for repeated tasks

Core Concepts Mastered:

1. **Thread Creation:** Functions, lambdas, member functions
2. **Synchronization:** `join()` vs `detach()` patterns
3. **Exception Safety:** RAII wrappers for automatic cleanup
4. **Move Semantics:** Efficient thread and resource transfer
5. **Runtime Optimization:** Hardware-aware thread configuration

Next Steps:

- Thread synchronization with mutexes and locks
- Condition variables for thread coordination
- Parallel algorithms in the standard library

<mutex>

<mutex>

Race Conditions

Race Condition Problem

The Core Issue: Multiple threads accessing shared data

```
int counter = 0; // Shared variable

void increment_unsafe(int iterations) {
    for(int i = 0; i < iterations; i++) {
        counter++; // NOT thread-safe!
    }
}
```

```
// 4 threads, each incrementing 100000 times
std::vector<std::thread> threads;
for(int i = 0; i < 4; i++) {
    threads.emplace_back(increment_unsafe, 100000);
}
for(auto& t : threads) { t.join(); }

// Expected: 400000, Actual: varies (e.g., 387234)
```

Why Race Conditions Happen

Assembly breakdown of counter++:

```
mov eax, [counter] // Read
add eax, 1         // Modify
mov [counter], eax // Write
```

Thread Interleaving Problem:

- Thread 1 reads counter = 0
- Thread 2 reads counter = 0
- Thread 1 writes counter = 1
- Thread 2 writes counter = 1
- **Lost update!** Should be 2

Critical Section: Code accessing shared resources

<mutex>

Protecting Shared Data with Mutexes

Using `std::mutex` with RAI:

```
std::mutex counter_mutex;  
int counter = 0;  
  
void increment_safe(int iterations) {  
    for(int i = 0; i < iterations; i++) {  
        std::lock_guard<std::mutex> lock(counter_mutex);  
        counter++; // Now thread-safe!  
        // Automatic unlock when lock goes out of scope  
    }  
}
```

RAII Benefits:

- Exception safety
- Automatic cleanup
- No manual lock/unlock

Class-based Protection:

```
class ThreadSafeCounter {  
private:  
    int count = 0;  
    mutable std::mutex mtx;  
  
public:  
    void increment() {  
        std::lock_guard<std::mutex> lock(mtx);  
        ++count;  
    }  
  
    int get() const {  
        std::lock_guard<std::mutex> lock(mtx);  
        return count;  
    }  
};
```

Key Principle: Keep data and mutex together

<mutex>

Deadlock Prevention

The Deadlock Problem

Classic Deadlock Scenario:

```
std::mutex mutex1, mutex2;

void thread1() {
    std::lock_guard<std::mutex> lock1(mutex1);
    // ... some work ...
    std::lock_guard<std::mutex> lock2(mutex2); // Waits forever
}

void thread2() {
    std::lock_guard<std::mutex> lock2(mutex2);
    // ... some work ...
    std::lock_guard<std::mutex> lock1(mutex1); // Waits forever
}
```

Result: Circular wait → Deadlock!

Strategy 1: Ordered Locking

```
// Always lock in same order
std::lock_guard<std::mutex> lock1(mutex1); // First
std::lock_guard<std::mutex> lock2(mutex2); // Second
```

Strategy 2: Atomic Multi-lock

```
std::unique_lock<std::mutex> lock1(mutex1, std::defer_lock);
std::unique_lock<std::mutex> lock2(mutex2, std::defer_lock);
std::lock(lock1, lock2); // Atomic acquisition
```

Strategy 3: Address-based Ordering

```
if (&mutex1 < &mutex2) {  
    std::lock_guard<std::mutex> lock1(mutex1);  
    std::lock_guard<std::mutex> lock2(mutex2);  
} else {  
    std::lock_guard<std::mutex> lock2(mutex2);  
    std::lock_guard<std::mutex> lock1(mutex1);  
}
```

Practical Example: Bank Transfer

```
class BankAccount {
    double balance;
    mutable std::mutex mtx;
public:
    static void transfer(BankAccount& from, BankAccount& to,
                        double amount) {
        // Prevent deadlock with consistent ordering
        if (&from < &to) {
            std::lock_guard<std::mutex> lock1(from.mtx);
            std::lock_guard<std::mutex> lock2(to.mtx);
        } else {
            std::lock_guard<std::mutex> lock1(to.mtx);
            std::lock_guard<std::mutex> lock2(from.mtx);
        }
        from.balance -= amount;
        to.balance += amount;
    }
};
```

<mutex>

Key Takeaways

1. **Use RAII:** `std::lock_guard` over manual lock/unlock
2. **Keep critical sections short:** Minimize lock duration
3. **Consistent lock ordering:** Prevent deadlocks
4. **Encapsulate:** Keep data and mutex together
5. **Avoid nested locks:** When possible

- `std::mutex` - Basic mutual exclusion
- `std::lock_guard` - RAII lock wrapper
- `std::unique_lock` - Flexible locking
- `std::lock()` - Multi-mutex atomic locking

<condition_variable>

<condition_variable>

Waiting for an Event or Other Condition

The Problem with Polling

Inefficient Waiting with `sleep_for`:

```
bool data_ready = false;
std::mutex data_mutex;

void consumer() {
    while (true) {
        {
            std::lock_guard<std::mutex> lock(data_mutex);
            if (data_ready) { /* Process data */
                data_ready = false; break;
            }
        }
        std::this_thread::sleep_for(std::chrono::milliseconds(100));
        // Wastes CPU cycles and introduces latency!
    }
}
```

- CPU wastage from constant polling
- Unnecessary latency, unpredictable timing

Better Approach: Wait for notification instead of polling

```
std::mutex m;
std::condition_variable cv;
bool data_ready = false;
void producer() { /* Prepare data...*/
    {
        std::lock_guard<std::mutex> lock(m);
        data_ready = true;
    }
    cv.notify_one(); // Wake up waiting thread
}

void consumer() {
    std::unique_lock<std::mutex> lock(m);
    cv.wait(lock, []{return data_ready;});
    // Data is ready, process it
}
```

Benefits: Efficient waiting, immediate response, no CPU waste

<condition_variable>

Waiting for a Condition with Condition Variables

Two Implementations Available:

- `std::condition_variable`: Works only with `std::mutex`
- `std::condition_variable_any`: Works with any mutex type (extra overhead)

Condition Variable Basics

```
std::mutex m;
std::condition_variable cv;
bool condition = false;

void waiter() {
    std::unique_lock<std::mutex> lock(m);
    cv.wait(lock, []{return condition;});
    // Condition is now true
}

void notifier() {
    {
        std::lock_guard<std::mutex> lock(m);
        condition = true;
    }
    cv.notify_one(); // or notify_all()
}
```

Understanding wait() Overloads

Two versions of wait():

```
// Version 1: Basic wait
void wait(std::unique_lock<std::mutex>& lock);

// Version 2: Wait with predicate (recommended)
template<class Predicate>
void wait(std::unique_lock<std::mutex>& lock, Predicate pred);
```

Predicate version equivalent to:

```
while (!pred()) wait(lock); // Handles spurious wakeups
```

How wait() Works Internally

1. Atomically releases the mutex
2. Blocks current thread
3. Waits for notification
4. Re-acquires mutex when awakened
5. Checks predicate (if provided)

Why unique_lock Instead of lock_guard?

Condition variables need flexible locking:

```
// This WON'T work - lock_guard can't be unlocked
std::lock_guard<std::mutex> lock(m);
cv.wait(lock); // Compiler error!

// This WORKS - unique_lock supports unlock/relock
std::unique_lock<std::mutex> lock(m);
cv.wait(lock); // Can unlock and relock automatically
```

Key Difference:

- lock_guard: RAII-only, cannot be manually unlocked
- unique_lock: Flexible, supports deferred/conditional locking

Two ways to notify waiting threads:

```
std::condition_variable cv;  
  
// Wake up ONE waiting thread  
cv.notify_one();  
  
// Wake up ALL waiting threads  
cv.notify_all();
```

When to use each:

- `notify_one()`: When only one thread should process the event
- `notify_all()`: When multiple threads need to check the condition

Best Practice: Always modify shared state before notifying

<condition_variable>

Building a Thread-Safe Queue with Condition Variables

Requirements:

- Multiple producers can push safely
- Multiple consumers can pop safely
- Consumers wait when queue is empty
- No busy waiting or polling

Thread-Safe Queue Design

Core Components:

```
template<typename T>
class ThreadSafeQueue {
private:
    std::queue<T> queue_;
    mutable std::mutex mutex_;
    std::condition_variable condition_;

public:
    void push(const T& item);
    T pop();
    bool empty() const;
    size_t size() const;
};
```

Implementation: Push Operation

Producer side - Adding items:

```
template<typename T>
void ThreadSafeQueue<T>::push(const T& item) {
    { std::lock_guard<std::mutex> lock(mutex_);
      queue_.push(item); }
    condition_.notify_one(); // Wake up waiting consumer
}

template<typename T>
void ThreadSafeQueue<T>::push(T&& item) {
    { std::lock_guard<std::mutex> lock(mutex_);
      queue_.push(std::move(item)); }
    condition_.notify_one();
}
```

Key Points:

- Lock only while modifying queue
- Notify after releasing lock

Implementation: Pop Operation

Consumer side - Retrieving items:

```
template<typename T>
T ThreadSafeQueue<T>::pop() {
    std::unique_lock<std::mutex> lock(mutex_);

    // Wait until queue is not empty
    condition_.wait(lock, [this] {
        return !queue_.empty();
    });

    T item = std::move(queue_.front());
    queue_.pop();
    return item;
}
```

Implementation: Pop Operation

Alternative with timeout:

```
template<typename T>
bool ThreadSafeQueue<T>::try_pop(T& item,
                                std::chrono::milliseconds timeout) {
    std::unique_lock<std::mutex> lock(mutex_);
    if (condition_.wait_for(lock, timeout,
                            [this] { return !queue_.empty(); })) {
        item = std::move(queue_.front());
        queue_.pop();
        return true;
    }
    return false; // Timeout occurred
}
```

Complete Usage Example

Producer - Consumer with Thread-Safe Queue:

```
ThreadSafeQueue<int> queue;
void producer(int start, int count) {
    for (int i = start; i < start + count; ++i) {
        queue.push(i);
        std::this_thread::sleep_for(std::chrono::milliseconds(10));
    }
}
void consumer(int id) {
    for (int i = 0; i < 5; ++i) {
        int value = queue.pop(); // Blocks until item available
        std::cout << "Consumer " << id << " got: " << value << std::endl;
    }
}
int main() {
    std::thread prod1(producer, 0, 10);
    std::thread cons1(consumer, 1);
    std::thread cons2(consumer, 2);
    prod1.join(); cons1.join(); cons2.join();
    return 0;
}
```

<condition_variable>

Key Takeaways

Essential Guidelines:

1. **Always use predicates:** Protects against spurious wakeups
2. **Use `unique_lock`:** Required for condition variable flexibility
3. **Modify state before notify:** Ensure consistency
4. **Keep critical sections short:** Minimize lock contention
5. **Choose notify method wisely:** `notify_one()` vs `notify_all()`

Common Patterns:

- Producer-Consumer queues
- Event signaling
- Worker thread pools
- Barrier synchronization

Remember: Condition variables are more efficient than polling for event-driven synchronization!

Parallel Algorithms

Parallel Algorithms

Parallelizing the Standard Library Algorithms

Enhancement to STL Algorithms: Parallel execution support

```
#include <execution>

std::vector<int> my_data = {3, 1, 4, 1, 5, 9, 2, 6};

// Serial execution (C++11/14)
std::sort(my_data.begin(), my_data.end());

// Parallel execution (C++17)
std::sort(std::execution::par, my_data.begin(), my_data.end());
```

Key Change: Additional first parameter for execution policy

Available for: `std::sort`, `std::for_each`, `std::transform`,
`std::reduce`, and many more!

Similarity to OpenMP

```
#pragma omp parallel for
for(unsigned i=0;i<my_data.size();++i)
{
    do_stuff(my_data[i]);
}

std::for_each(std::execution::par, my_data.begin(), my_data.end(), do_stuff);
```

Parallel Algorithms

Execution Policies

Execution Policy Types

Four execution policies available:

```
// 1. Sequential (same as no policy)
std::execution::sequenced_policy    // std::execution::seq
// 2. Parallel
std::execution::parallel_policy     // std::execution::par
// 3. Parallel + Vectorized
std::execution::parallel_unsequenced_policy // std::execution::par_unseq
// 4. Unsequenced (C++20)
std::execution::unsequenced_policy  // std::execution::unseq
std::sort(std::execution::seq, v.begin(), v.end());
std::sort(std::execution::par, v.begin(), v.end());
std::sort(std::execution::par_unseq, v.begin(), v.end());
```

Execution policies is a **permission**, not a **requirement**.

`std::execution::sequenced_policy:`

- Same as traditional serial execution
- Single thread, sequential order

`std::execution::parallel_policy:`

- Multiple threads allowed
- Operations may be interleaved
- No vectorization guarantees

`std::execution::parallel_unsequenced_policy:`

- Multiple threads + vectorization
- Operations may execute in any order
- Best performance potential
- Requires thread-safe operations

Parallel Algorithms

The Parallel Algorithms from the C++ Standard Library

Examples of Using Parallel Algorithms

Parallel Transform:

```
std::vector<int> input = {1, 2, 3, 4, 5};  
std::vector<int> output(input.size());  
  
std::transform(std::execution::par,  
               input.begin(), input.end(),  
               output.begin(),  
               [](int x) { return x * x; });
```

Examples of Using Parallel Algorithms

Parallel For Each:

```
std::vector<std::string> words = {"hello", "world", "parallel"};

std::for_each(std::execution::par,
              words.begin(), words.end(),
              [](std::string& word) {
                  std::transform(word.begin(), word.end(),
                                word.begin(), ::toupper);
              });
```

Examples of Using Parallel Algorithms

Parallel Reduction:

```
std::vector<int> numbers(1000000);  
std::iota(numbers.begin(), numbers.end(), 1);  
  
// Parallel sum  
auto sum = std::reduce(std::execution::par,  
                      numbers.begin(), numbers.end(),  
                      0);  
  
// Parallel sum with custom operation  
auto product = std::reduce(std::execution::par,  
                          numbers.begin(), numbers.end(),  
                          1,  
                          std::multiplies<int>());
```

Examples of Using Parallel Algorithms

Transform Reduce:

```
auto squared_sum = std::transform_reduce(
    std::execution::par,
    numbers.begin(), numbers.end(),
    0,
    std::plus<>(),
    [](int x) { return x * x; }
);
```

Counting Visits Example

Website Analytics - Parallel Processing:

```
struct Visit {
    std::string page;
    int user_id;

    ↪ std::chrono::time_point<std::chrono::system_clock>
    ↪ timestamp;
};

std::vector<Visit> visits = load_visits();

// Count unique pages visited in parallel
std::sort(std::execution::par, visits.begin(),
    ↪ visits.end(),
    [](const Visit& a, const Visit& b) {
        return a.page < b.page;
    });
```

```
// Count visits per page
std::map<std::string, int> page_counts;
auto current = visits.begin();
while (current != visits.end()) {
    auto range_end = std::upper_bound(
        std::execution::par,
        current, visits.end(), *current,
        [](const Visit& a, const Visit& b) {
            return a.page < b.page;
        });

    page_counts[current->page] =
        std::distance(current, range_end);
    current = range_end;
}
```

When to Use Parallel Algorithms:

- Large datasets (typically > 10,000 elements)
- CPU-intensive operations
- Independent computations

Overhead Example:

```
// Small dataset - sequential might be faster
std::vector<int> small_data(100);
std::sort(std::execution::seq, small_data.begin(), small_data.end());

// Large dataset - parallel beneficial
std::vector<int> large_data(1000000);
std::sort(std::execution::par, large_data.begin(), large_data.end());
```

Thread Safety Requirements:

- Operations must be thread-safe for `par` and `par_unseq`
- Avoid shared mutable state
- Use atomic operations when necessary

Parallel Algorithms

Assignment: GPU Execution with NVIDIA

Homework Assignment

Research Task:

1. Review NVIDIA compiler manual (nvc++)
2. Study how to run C++ standard algorithms on GPU
3. Observe CUDA calls generated by the compiler

Experiment Steps:

1. Load NVIDIA HPC SDK
2. Compile parallel algorithms with GPU support
3. Use correct compile option
4. Analyze generated CUDA code

Expected Results:

- Standard C++ code runs without modification
- GPU kernel functions are auto-generated
- Significant performance improvement observed

Tip: Use nvprof or nsys for performance analysis

Assignment Example Code

Compile Command: `nvc++ <correct_option> -O3 test.cpp -o test`

Test Code Template:

```
const size_t N = 10'000'000;
std::vector<int> data(N);
std::iota(data.begin(), data.end(), 1);
std::random_shuffle(data.begin(), data.end());

auto start = std::chrono::high_resolution_clock::now();

// This should run on GPU with nvc++ and correct options
std::sort(std::execution::par_unseq,
          data.begin(), data.end());

auto end = std::chrono::high_resolution_clock::now();
auto duration = std::chrono::duration_cast<
    std::chrono::milliseconds>(end - start);

std::cout << "Sorting took: " << duration.count()
          << " ms" << std::endl;
```

Summary

- **Object-Oriented Programming:** Classes, inheritance, polymorphism
- **RAII (Resource Acquisition Is Initialization):** Automatic resource management
- **Move Semantics:** Efficient resource transfer with move constructors and assignment
- **Smart Pointers:** `unique_ptr`, `shared_ptr` for memory safety
- **Modern C++ Features:** Auto type deduction, range-based for loops, lambdas
- **Standard Template Library (STL):** Containers, algorithms, iterators

- **Thread Management:** Creating, joining, and detaching threads
- **Mutual Exclusion:** `std::mutex`, `std::lock_guard`, `std::unique_lock`
- **Thread Communication:** `std::condition_variable` for signaling
- **Parallel Algorithms:** STL algorithms with execution policies
- **Atomic Operations:** Lock-free programming with `std::atomic`
- **Memory Ordering:** Understanding memory consistency models

Best Practices for Thread Management

- **RAII for Locks:** Always use lock guards, avoid manual lock/unlock
- **Minimize Shared State:** Reduce data sharing between threads
- **Prefer Higher-Level Abstractions:** Use parallel algorithms over raw threads
- **Avoid Deadlocks:** Consistent lock ordering, timeout mechanisms
- **Exception Safety:** Ensure proper cleanup in exception scenarios
- **Thread-Local Storage:** Use for thread-specific data

Framework Comparison: OpenMP vs C++ Threads vs MPI

Aspect	OpenMP	C++ Threads	MPI
Learning Curve	Low	Medium	High
Development Time	Fast	Medium	Slow
Fine-grained Control	Limited	High	High
Memory Model	Shared	Shared	Distributed
Scalability	Node-level	Node-level	Cluster-level
Debugging	Easy	Medium	Hard

When to Choose Which Framework

- **OpenMP:**

- Quick parallelization of existing code
- Loop-based parallelism
- Prototyping and simple parallel tasks

- **C++ Threads:**

- Complex synchronization requirements
- Object-oriented concurrent design
- Integration with modern C++ features

- **MPI:**

- Distributed computing across multiple nodes
- Large-scale scientific computing
- Heterogeneous computing environments

Key Takeaways

- **Concurrency is Complex:** Start simple, understand fundamentals first
- **Choose the Right Tool:** Match the framework to your problem domain
- **Safety First:** Use RAI, smart pointers, and high-level abstractions
- **Performance Matters:** Profile and measure, don't guess
- **Modern C++ Advantage:** Combines performance with safety and expressiveness

Next Steps for Further Study

- **Advanced Topics:** Lock-free programming, memory models, coroutines
- **Libraries:** Intel TBB, Boost.Thread, OpenCL for GPU computing
- **Design Patterns:** Producer-consumer, thread pools, actor models
- **Performance Analysis:** Profiling tools, bottleneck identification
- **Real-world Projects:** Apply concepts to practical problems

THANK You

ANY QUESTIONS?